

Sample Table: Employees

emp_id	emp_name	department	salary
101	Saran	IT	55000
102	Priya	HR	45000
103	Rahul	IT	65000
104	Kumar	Finance	70000
105	Anjali	HR	60000

```
CREATE TABLE Employees (
```

```
    emp_id INT PRIMARY KEY,
```

```
    emp_name VARCHAR(50),
```

```
    department VARCHAR(20),
```

```
    salary INT);
```

```
INSERT INTO Employees VALUES
```

```
(101,'Saran','IT',55000),
```

```
(102,'Priya','HR',45000),
```

```
(103,'Rahul','IT',65000),
```

```
(104,'Kumar','Finance',70000),
```

```
(105,'Anjali','HR',60000);
```

1. Simple Stored Procedure (5 Tasks)

Task 1

Create a stored procedure to display all employees.

```
DELIMITER //
```

```
CREATE PROCEDURE DisplayAllEmployees()
```

```
BEGIN
```

```
    SELECT * FROM Employees;
```

```
END //
```

```
DELIMITER ;
```

```
call DisplayAllEmployees();
```

Task 2

Create a stored procedure to display employee names and salaries.

DELIMITER //

CREATE PROCEDURE DisplayNameSalary()

BEGIN

 SELECT emp_name, salary FROM Employees;

END //

DELIMITER ;

call DisplayNameSalary();

Task 3

Create a stored procedure to display all departments.

DELIMITER //

CREATE PROCEDURE DisplayDepartments()

BEGIN

 SELECT DISTINCT department FROM Employees;

END //

DELIMITER ;

call DisplayDepartments();

Task 4

Create a stored procedure to display employees in ascending order of salary.

DELIMITER //

CREATE PROCEDURE SalaryAscending()

BEGIN

 SELECT * FROM Employees ORDER BY salary ASC;

END //

DELIMITER ;

call SalaryAscending();

Task 5

Create a stored procedure to display the total number of employees.

DELIMITER //

CREATE PROCEDURE EmployeeCount()

BEGIN

 SELECT COUNT(*) AS TotalEmployees FROM Employees;

END //

DELIMITER ;

call EmployeeCount();

2. Procedure with WHERE Condition (5 Tasks)

Task 6

Create a procedure to display employees working in the IT department.

DELIMITER //

CREATE PROCEDURE ITEmployees()

BEGIN

 SELECT * FROM Employees WHERE department='IT';

END //

DELIMITER ;

call ITEmployees();

Task 7

Create a procedure to display employees whose salary is greater than ₹50,000.

DELIMITER //

CREATE PROCEDURE SalaryAbove50000()

BEGIN

 SELECT * FROM Employees WHERE salary > 50000;

END //

DELIMITER ;

call SalaryAbove50000();

Task 8

Create a procedure to display employees belonging to the HR department.

DELIMITER //

CREATE PROCEDURE HREmployees()

BEGIN

 SELECT * FROM Employees WHERE department='HR';

END //

DELIMITER ;

call HREmployees();

Task 9

Create a procedure to display employees earning between ₹45,000 and ₹65,000.

DELIMITER //

CREATE PROCEDURE SalaryRange()

BEGIN

 SELECT * FROM Employees

 WHERE salary BETWEEN 45000 AND 65000;

END //

DELIMITER ;

call SalaryRange();

Task 10

Create a procedure to display employees whose names start with 'S'.

DELIMITER //

CREATE PROCEDURE NamesStartingS()

BEGIN

 SELECT * FROM Employees WHERE emp_name LIKE 'S%';

END //

DELIMITER ;

call NamesStartingS();

3. Procedure with Input Parameter (5 Tasks)

Task 11

Create a procedure that accepts a department name and displays employees from that department.

```
DELIMITER //
```

```
CREATE PROCEDURE EmployeesByDepartment(IN dept VARCHAR(20))
```

```
BEGIN
```

```
    SELECT * FROM Employees WHERE department=dept;
```

```
END //
```

```
DELIMITER ;
```

```
call EmployeesByDepartment("IT");
```

Task 12

Create a procedure that accepts an employee ID and displays employee details.

```
DELIMITER //
```

```
CREATE PROCEDURE EmployeeByID(IN eid INT)
```

```
BEGIN
```

```
    SELECT * FROM Employees WHERE emp_id=eid;
```

```
END //
```

```
DELIMITER ;
```

```
call EmployeeByID(103);
```

Task 13

Create a procedure that accepts a salary and displays employees earning more than that amount

```
DELIMITER //
```

```
CREATE PROCEDURE SalaryGreaterThan(IN sal INT)
```

```
BEGIN
```

```
    SELECT * FROM Employees WHERE salary > sal;
```

```
END //
```

DELIMITER ;

call SalaryGreaterThan(47000);

Task 14

Create a procedure that accepts an employee name and displays the corresponding employee record.

DELIMITER //

CREATE PROCEDURE EmployeeByName(IN ename VARCHAR(50))

BEGIN

 SELECT * FROM Employees WHERE emp_name=ename;

END //

DELIMITER ;

call EmployeeByName("Priya");

Task 15

Create a procedure that accepts a department and displays the number of employees in that department

DELIMITER //

CREATE PROCEDURE DepartmentCount(IN dept VARCHAR(20))

BEGIN

 SELECT COUNT(*) AS Total FROM Employees WHERE department=dept;

END //

DELIMITER ;

call DepartmentCount("HR");

4. Procedure with Multiple Input Parameters (5 Tasks)

Task 16

Create a procedure that accepts department and salary and displays employees satisfying both conditions

```
DELIMITER //

CREATE PROCEDURE DeptAndSalary(
IN dept VARCHAR(20),
IN sal INT)
BEGIN
    SELECT * FROM Employees WHERE department=dept AND salary>=sal;
END //

DELIMITER ;

call DeptAndSalary("IT",48000);
```

Task 17

Create a procedure that accepts minimum salary and maximum salary and displays employees within that range.

```
DELIMITER //

CREATE PROCEDURE SalaryBetween(
IN minsal INT,
IN maxsal INT)
BEGIN
    SELECT * FROM Employees WHERE salary BETWEEN minsal AND maxsal;
END //

DELIMITER ;

call SalaryBetween(45000,70000);
```

Task 18

Create a procedure that accepts employee ID and department and displays the matching employee.

```
DELIMITER //

CREATE PROCEDURE IDAndDept(
IN eid INT,
IN dept VARCHAR(20))
```

```
BEGIN

    SELECT * FROM Employees WHERE emp_id=eid AND department=dept;

END //

DELIMITER ;

call IDAndDept(103,"HR");
```

Task 19

Create a procedure that accepts employee name and salary and displays the matching employee.

```
DELIMITER //

CREATE PROCEDURE NameAndSalary(

    IN ename VARCHAR(50),

    IN sal INT)

BEGIN

    SELECT * FROM Employees WHERE emp_name=ename AND salary=sal;

END //

DELIMITER ;

call NameAndSalary("Rahul",48000);
```

Task 20

Create a procedure that accepts department and employee name and displays the employee details.

```
DELIMITER //

CREATE PROCEDURE DeptAndName(

    IN dept VARCHAR(20),

    IN ename VARCHAR(50))

BEGIN

    SELECT * FROM Employees

    WHERE department=dept AND emp_name=ename;

END //
```

DELIMITER ;

call DeptAndName("HR","Saran");

5. Procedure for INSERT (5 Tasks)

Task 21

Create a procedure to insert a new employee record.

DELIMITER //

CREATE PROCEDURE InsertEmployee()

BEGIN

INSERT INTO Employees VALUES (106,'Sneha','Sales',50000);

END //

DELIMITER ;

CALL InsertEmployee();

Task 22

Create a procedure that inserts employee ID, employee name, department, and salary using input parameters.

DELIMITER //

CREATE PROCEDURE InsertEmployeeDetails(

IN eid INT,

IN ename VARCHAR(50),

IN dept VARCHAR(20),

IN sal INT)

BEGIN

INSERT INTO Employees VALUES(eid,ename,dept,sal);

END //

DELIMITER ;

CALL InsertEmployeeDetails(107,'Arjun','IT',62000);

Task 23

Create a procedure to add an employee to the HR department.

```
DELIMITER //

CREATE PROCEDURE AddHREmployee(
  IN eid INT,
  IN ename VARCHAR(50),
  IN sal INT)
BEGIN
  INSERT INTO Employees VALUES(eid,ename,'HR',sal);
END //

DELIMITER ;

CALL AddHREmployee(108,'Deepa',55000);
```

Task 24

Create a procedure to insert multiple employee records by calling the procedure repeatedly.

```
DELIMITER //

CREATE PROCEDURE InsertEmployeeDetails(
  IN eid INT,
  IN ename VARCHAR(50),
  IN dept VARCHAR(20),
  IN sal INT)
BEGIN
  INSERT INTO Employees VALUES(eid,ename,dept,sal);
END //

DELIMITER ;

CALL InsertEmployeeDetails(109,'Ravi','IT',60000);

CALL InsertEmployeeDetails(110,'Kiran','Finance',58000);

CALL InsertEmployeeDetails(111,'Pooja','HR',52000);
```

Task 25

Create a procedure to insert a record and then display all employees.

```
DELIMITER //

CREATE PROCEDURE InsertAndDisplay()

BEGIN

    INSERT INTO Employees VALUES(112,'Meena','Sales',54000);

    SELECT * FROM Employees;

END //

DELIMITER ;

CALL InsertAndDisplay();
```

6. Procedure for UPDATE (5 Tasks)

Task 26

Create a procedure to update the salary of an employee using employee ID.

```
DELIMITER //

CREATE PROCEDURE UpdateSalary(

IN eid INT,

IN newsalary INT)

BEGIN

    UPDATE Employees SET salary=newsalary WHERE emp_id=eid;

END //

DELIMITER ;

CALL UpdateSalary(101,60000);
```

Task 27

Create a procedure to update the department of an employee.

```
DELIMITER //

CREATE PROCEDURE UpdateDepartment(

IN eid INT,

IN newdept VARCHAR(20))

BEGIN

    UPDATE Employees SET department=newdept WHERE emp_id=eid;
```

END //

DELIMITER ;

CALL UpdateDepartment(102,'Finance');

Task 28

Create a procedure to increase the salary of all employees in the IT department by ₹5,000.

DELIMITER //

CREATE PROCEDURE IncreaseITSalary()

BEGIN

 UPDATE Employees SET salary=salary+5000 WHERE department='IT';

END //

DELIMITER ;

CALL IncreaseITSalary();

Task 29

Create a procedure that accepts employee ID and new salary and updates the employee record.

DELIMITER //

CREATE PROCEDURE UpdateEmployeeSalary(

IN eid INT,

IN sal INT)

BEGIN

 UPDATE Employees SET salary=sal WHERE emp_id=eid;

END //

DELIMITER ;

CALL UpdateEmployeeSalary(103,70000);

Task 30

Create a procedure that updates employee names using employee ID.

```
DELIMITER //

CREATE PROCEDURE UpdateEmployeeName(
IN eid INT,
IN newname VARCHAR(50))
BEGIN
    UPDATE Employees SET emp_name=newname WHERE emp_id=eid;
END //

DELIMITER ;

CALL UpdateEmployeeName(104,'Vikram');
```

7. Procedure for DELETE (5 Tasks)

Task 31

Create a procedure to delete an employee using employee ID.

```
DELIMITER //

CREATE PROCEDURE DeleteEmployee(
IN eid INT)
BEGIN
    DELETE FROM Employees WHERE emp_id=eid;
END //

DELIMITER ;

CALL DeleteEmployee(105);
```

Task 32

Create a procedure to delete all employees belonging to the HR department.

```
DELIMITER //

CREATE PROCEDURE DeleteHREmployees()
BEGIN
    DELETE FROM Employees
    WHERE department='HR';
END //
```

DELIMITER ;

CALL DeleteHREmployees();

Task 33

Create a procedure to delete employees whose salary is less than ₹50,000.

DELIMITER //

CREATE PROCEDURE DeleteLowSalary()

BEGIN

DELETE FROM Employees WHERE salary<50000;

END //

DELIMITER ;

CALL DeleteLowSalary();

Task 34

Create a procedure that accepts employee name and deletes the corresponding record.

DELIMITER //

CREATE PROCEDURE DeleteEmployeeByName(

IN ename VARCHAR(50))

BEGIN

DELETE FROM Employees WHERE emp_name=ename;

END //

DELIMITER ;

CALL DeleteEmployeeByName('Priya');

Task 35

Create a procedure to delete an employee and then display the remaining records.

DELIMITER //

CREATE PROCEDURE DeleteAndDisplay(

IN eid INT)

BEGIN

DELETE FROM Employees WHERE emp_id=eid;

```
SELECT * FROM Employees;

END //

DELIMITER ;

CALL DeleteAndDisplay(101);
```

Interview-Level Bonus Tasks

Task 36

Create a procedure to display the highest-paid employee.

```
DELIMITER //

CREATE PROCEDURE HighestPaidEmployee()

BEGIN

    SELECT * FROM Employees WHERE salary=(SELECT MAX(salary) FROM Employees);

END //

DELIMITER ;

CALL HighestPaidEmployee();
```

Task 37

Create a procedure to display the average salary department-wise.

```
DELIMITER //

CREATE PROCEDURE AvgSalaryByDepartment()

BEGIN

    SELECT department, AVG(salary) AS AverageSalary FROM Employees

    GROUP BY department;

END //

DELIMITER ;

CALL AvgSalaryByDepartment();
```

Task 38

Create a procedure to increase salary by 10% for a given department

```
DELIMITER //

CREATE PROCEDURE IncreaseDeptSalary(
```

```
IN dept VARCHAR(20))
BEGIN
    UPDATE Employees
    SET salary=salary*1.10
    WHERE department=dept;
END //
DELIMITER ;
CALL IncreaseDeptSalary('IT');
```

Task 39

Create a procedure that returns the employee count for a given department.

```
DELIMITER //
CREATE PROCEDURE EmployeeCountByDept(
IN dept VARCHAR(20))
BEGIN
    SELECT COUNT(*) AS TotalEmployees FROM Employees
    WHERE department=dept;
END
```

Task 40

Create a procedure to delete employees who are not assigned to any department.

```
DELIMITER //
CREATE PROCEDURE DeleteNoDepartment()
BEGIN
    DELETE FROM Employees WHERE department IS NULL
    OR department="";
END //
DELIMITER ;
CALL DeleteNoDepartment();'
```